

Arquitectura de Computadoras 2017

Taller de MIPS – parte 2
(cap. 2 del libro estructura y diseño de
computadores de Patterson y Hennessy)

Arquitectura MIPS

- Registros
- Formato de Instrucción
- Modos de Direccionamiento
- Juego de Instrucciones

Registros Visibles

- 32 registros de 32 bits
- Todas las operaciones aritméticas operan con registros
- Se identifican: \$0, \$1,....\$31
- \$0 está cableado a cero.

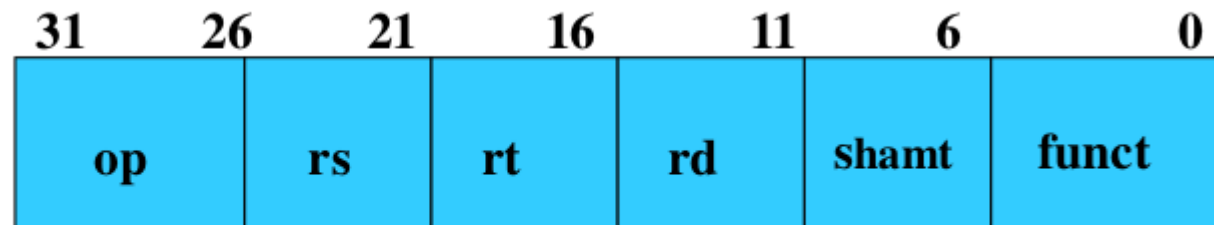
Identificación de registros

R0	[r0]	R8	[t0]	R16	[s0]	R24	[t8]
R1	[at]	R9	[t1]	R17	[s1]	R25	[t9]
R2	[v0]	R10	[t2]	R18	[s2]	R26	[k0]
R3	[v1]	R11	[t3]	R19	[s3]	R27	[k1]
R4	[a0]	R12	[t4]	R20	[s4]	R28	[gp]
R5	[a1]	R13	[t5]	R21	[s5]	R29	[sp]
R6	[a2]	R14	[t6]	R22	[s6]	R30	[s8]
R7	[a3]	R15	[t7]	R23	[s7]	R31	[ra]

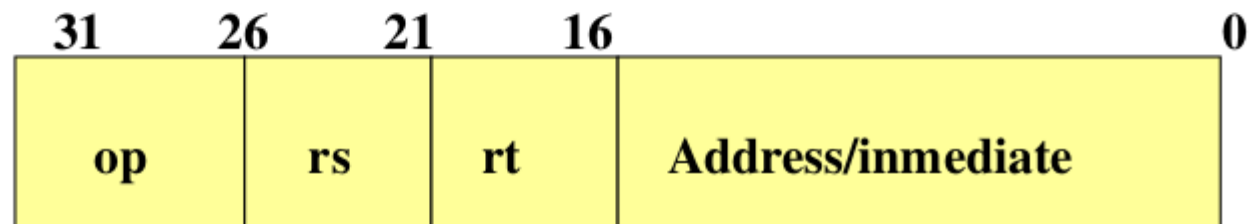
Formato MIPS de Instrucción

Son todas de 32 bits. Tres formatos:

- Tipo R



- Tipo I



- Tipo J



Formato R (Registro)



- Op= 0 (6 bits)
- rs, rt = identificación de los registros fuente (5 bits)
- rd = identificación del registro destino (5bits)
- shamt= cantidad de desplazamiento (5 bits)
- funct= identifica la operación (por ej. add=32, sub=34)
(6 bits)

Formato R :ejemplos

- Add \$1, \$2, \$3

31	26	21	16	11	6	0
0	2	3	1	0	32	

- Sub \$1, \$2, \$3

31	26	21	16	11	6	0
0	2	3	1	0	34	

- Slt \$1, \$2, \$3

31	26	21	16	11	6	0
0	2	3	1	0	42	

Set Less Than

si (\$2 < \$3) entonces \$1=1 sino \$1=0

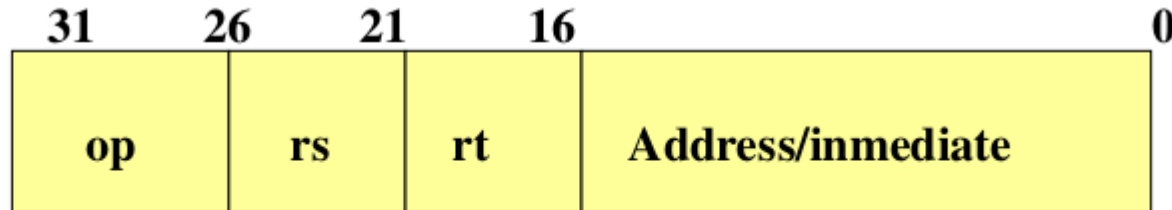
- Jr \$31

31	26	21	16	11	6	0
0	31	0	0	0	8	

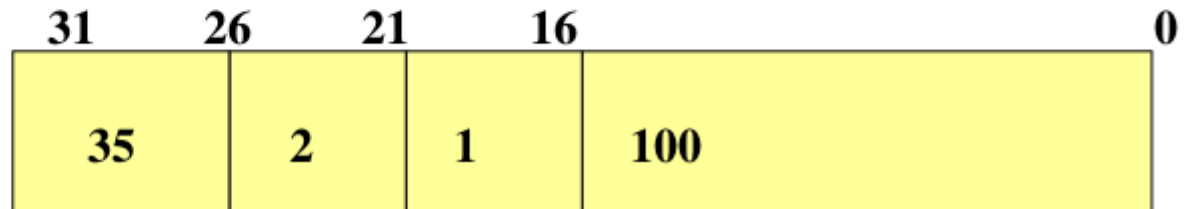
Jump Register

PC <-- \$31

Formato I : transferencias

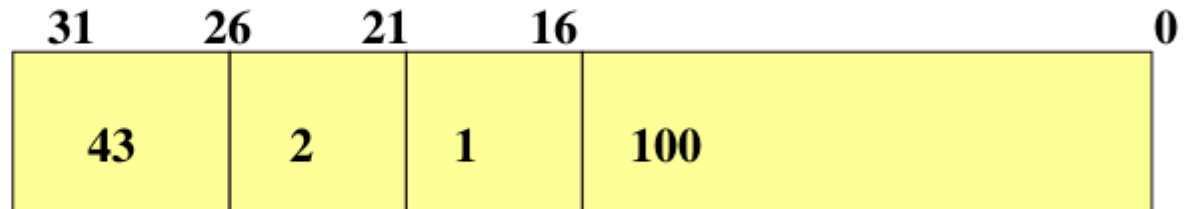


- Lw \$1, 100(\$2)



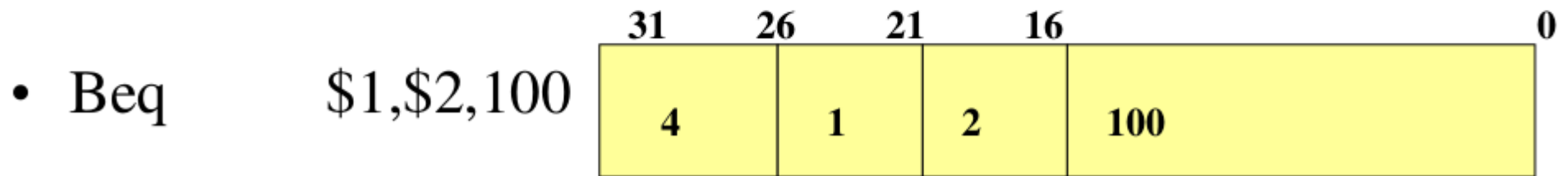
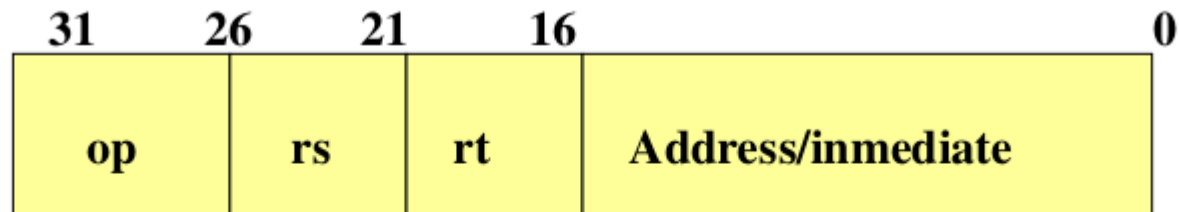
Load Word *$\$1 = M[\$2 + 100]$*

- Sw \$1, 100(\$2)

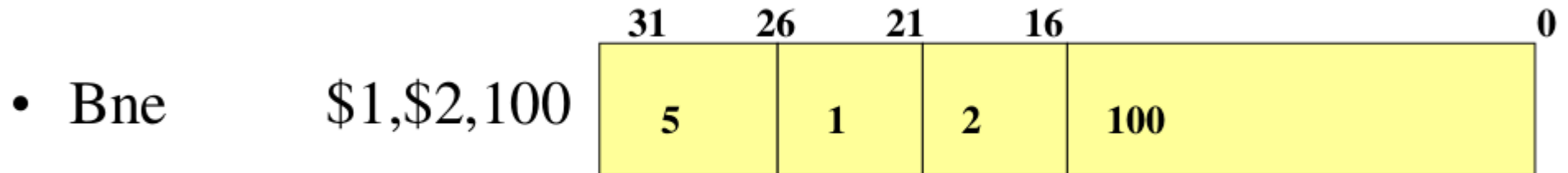


Store Word *$M[\$2 + 100] = \1*

Formato I : saltos condicionales

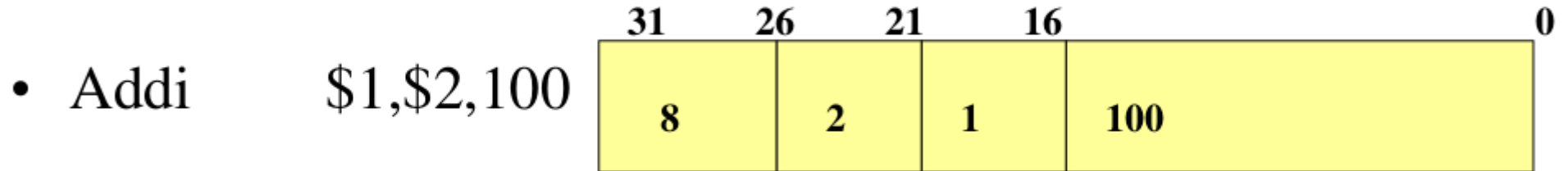
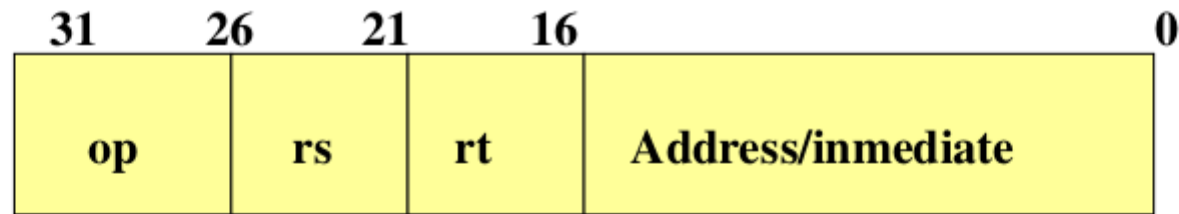


Branch Equal *si (\$1=\$2) entonces ir a PC+4+100*

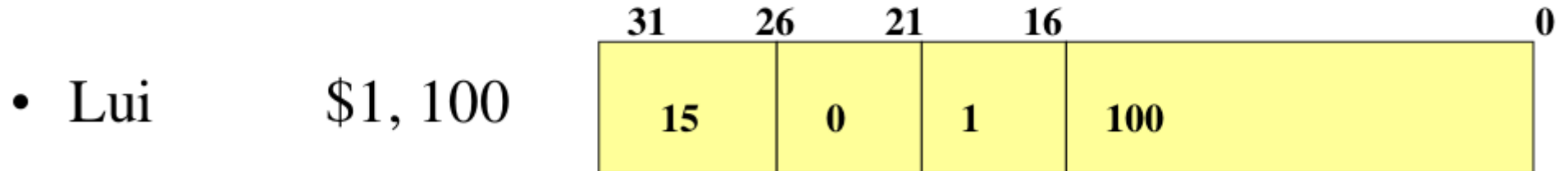


Branch Not Equal *si (\$1!= \$2) entonces ir a PC+4+100*

Formato I : transferencias inmediatas



$$\$1 = \$2 + 100$$



Load upper inmediato $\$1 = 100 * 2^{16}$

Formato J : saltos incondicionales



- J 10000



PC_{27..2}=10000 PC_{1..0}=00

- Jal 10000



JUMP AND LINK *\$31=PC+4*

PC_{27..2}=10000 PC_{1..0}=00

Ejemplo de direccionamiento

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: ...
```

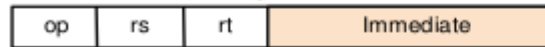
80000	0	0	19	9	4	0
80004	0	9	22	9	0	32
80008	35	9	8	0		
80012	5	8	21	2		
80016	8	19	19	1		
80020	2	20000				
80024						

Modos de Direcccionamiento

- Modo Registro
- Modo Base
- Modo Inmediato
- Modo Relativo al PC
- Modo Pseudodirecto

Modos de Direcccionamiento

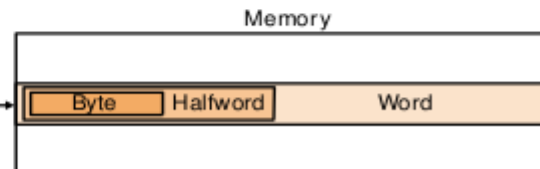
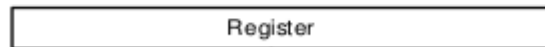
1. Immediate addressing



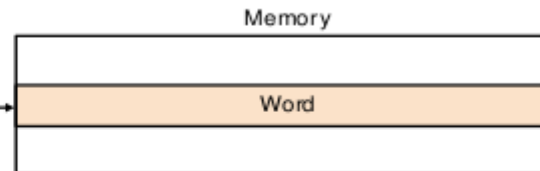
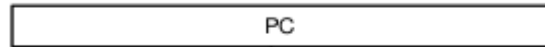
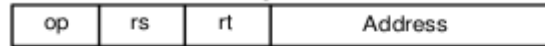
2. Register addressing



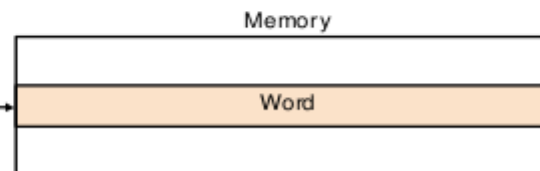
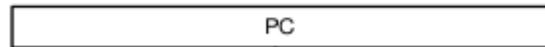
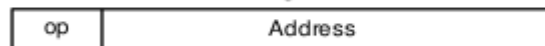
3. Base addressing



4. PC-relative addressing



5. Pseudodirect addressing



Modos de Direccionamiento (cont.)

1. **Direccionamiento Inmediato:** cuando el operando es una constante que aparece en la misma instrucción.
2. **Direccionamiento a Registro:** cuando el operando esta en un registro.
3. **Direccionamiento Base o Desplazamiento:** cuando el operando está en la posición de memoria cuya dirección es la suma de un registro y de una constante en la instrucción.
4. **Direccionamiento relativo al PC:** cuando la dirección es la suma del PC y de una constante en la instrucción.
5. **Direccionamiento pseudodirecto:** cuando la dirección de salto es los 26 bits de la instrucción concatenados con los bits de mayor peso del PC.

Ejemplo: Ensamblar

```
while (a[i]==k)
    i=i+j;
```

i, j, k están en \$s3, \$s4 y \$s5.
La dirección de a está en \$s6

```
bucle:
    add    $t1,$s3,$s3
    add    $t1,$t1,$t1    #$t1=4*i
    add    $t1,$t1,$s6
    lw     $t0,0($t1)
    bne    $t0,$s5, fin
    add    $s3,$s3,$s4
    j      bucle
fin:
```

Suponer que el bucle comienza
en la dirección 80000 de
memoria

Ejemplo: Ensamblar

80000	0	19	19	9	0	32
80004	0	9	9	9	0	32
80008	0	9	21	9	0	32
80012	35	9	8	0		
80016	5	8	21	2		
80020	0	19	20	19	0	32
80024	2	80000				

Cálculo de la dirección destino de salto

Branch Equal si (\$1=\$2) entonces ir a
PC+4+offset

El campo desplazamiento mide la cantidad de words (4 bytes) que separan el valor actual del PC de la dirección destino de salto

$$\textit{destino} = PC + 4 + (\textit{offset} * 4)$$

Ejemplos para correr en QtSpim

- a) Cargar dicho archivo en el simulador QTSPIM.
- b) ¿Qué posiciones de memoria ocupa el código que acaba de cargar? Y el resto del código que lo precede, ¿qué función cumple?
- c) Comprobar su funcionamiento utilizando el simulador QTSPIM utilizando la ejecución paso a paso (F10).

Ejemplos para correr en QtSpim(cont.): ejemplo1.asm

```
.data
A: .word 1, 3, 5, 7, 9
.text
main: addi $s3, $zero, 1 #i=1
      addi $s4, $zero, 1 #j=1
      addi $s5, $zero, 1 # k=1
      la $s6, A
Bucle: add $t1, $s3, $s3
      add $t1, $t1, $t1 # $t1=4*i
      add $t1, $t1, $s6
      lw $t0, 0($t1)
      bne $t0, $s5, Fin
      add $s3, $s3, $s4
      j Bucle
Fin: jr $ra
```

Ejemplos para correr en QtSpim(cont.): ejemplo2.asm

.data

MemArray : .word 2 ,3 ,5 ,7 ,11 ,13 ,17 ,19 ,23 ,29 ,31 ,37 ,41 ,43 ,47

.text

main : addi \$s0 , \$zero, 0

addi \$s2 , \$zero, 0

addi \$t1 , \$zero, 0

LOOP:lw \$s1 , MemArray (\$s0)

add \$s2 , \$s2 , \$s1

addi \$s0 , \$s0 , 4

addi \$t1 , \$t1 , 1

slti \$t2 , \$t1 , 6

bne \$t2 , \$zero , LOOP

jr \$ra